

Make at the center of your pipelines

WPipe vs. Luigi: Sustainable Orchestration for the Modern Dev

The context

Is your orchestrator consuming more energy and resources than your actual data logic? 🌱 In a world moving toward Green IT, efficiency isn't an option — it's a responsibility. Luigi was a pioneer, but its centralized architecture and high RAM usage belong to another era. For microservices, IoT, or agile pipelines, you need an engine that's powerful yet invisible.

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step
from wpipe import Parallel

@step(name="branch_a")
def branch_a(data): return {"a": data["x"]}

@step(name="branch_b")
def branch_b(data): return {"b": data["y"]}

pipe = Pipeline(pipeline_name="make_to_wpipe")
pipe.set_steps([Parallel(steps=[branch_a, branch_b])])
```

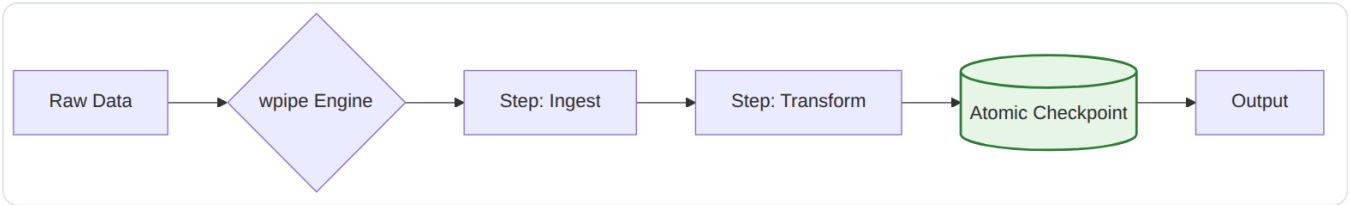
At a glance

Feature	WPipe	Luigi
RAM Footprint	< 50MB	> 2GB
Persistence	SQLite WAL (Sovereign)	Central Scheduler
Deployment	Zero-Infra (Library)	Server-Dependent

Feature	WPipe	Luigi
Green-IT	 High Efficiency	 Resource Heavy

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Make** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.